



[NextWork.org](https://nextwork.org)

VPC Peering

 ajv.singh3107@gmail.com

**ajv.singh3107@gmail.com**

NextWork Student

[NextWork.org](https://nextwork.org)

Introducing Today's Project!

What is Amazon VPC?

Amazon VPC is a service that enables you to create isolated networks in the AWS cloud. It's useful for controlling your network environment, including IP ranges, subnets, and security settings, ensuring secure, customizable config for your resources.

How I used Amazon VPC in this project

In today's project, I used Amazon VPC to create isolated networks for two EC2 instances. I configured VPC peering to enable communication between them, set up route tables for traffic flow, and adjusted security group rules to ensure proper access.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was the complexity of configuring security group rules for successful communication. I initially overlooked the need to allow ICMP traffic for the ping test, which added an extra troubleshooting step.

This project took me...

This project took me about 1 hour to complete. This included setting up the VPC peering, configuring the EC2 instances, troubleshooting connection issues, and ensuring successful communication between the instances.



ajv.singh3107@gmail.com

NextWork Student

[NextWork.org](https://nextwork.org)

In the first part of my project...

Step 1 - Set up my VPC

In this step, we'll create two VPCs from scratch using the visual VPC resource map for a quick and efficient setup. This tool will help us visualize and configure our VPCs, making the process faster and more straightforward.

Step 2 - Create a Peering Connection

In this step, we are setting VPC peering connection, allowing two VPCs to communicate securely without using the internet. After establishing the peering link, you'll update routing tables and security rules to enable traffic flow between the VPCs.

Step 3 - Update Route Tables

In this step, you're configuring routing between VPC 1 and VPC 2. You'll update the route tables in both VPCs to allow traffic from VPC 1 to reach VPC 2, and vice versa. This enables resources in each VPC to communicate with each other over the peering

Step 4 - Launch EC2 Instances

In this step, we're launching an EC2 instance in each VPC. These instances will be used to test the VPC peering connection later, ensuring that they can communicate with each other across the peering link by sending and receiving traffic.



ajv.singh3107@gmail.com
NextWork Student

[NextWork.org](https://nextwork.org)

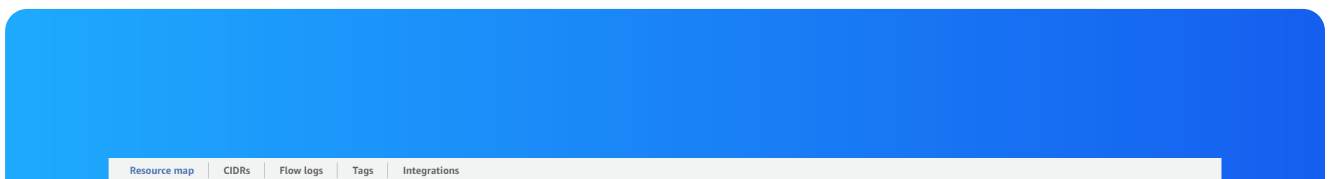
Multi-VPC Architecture

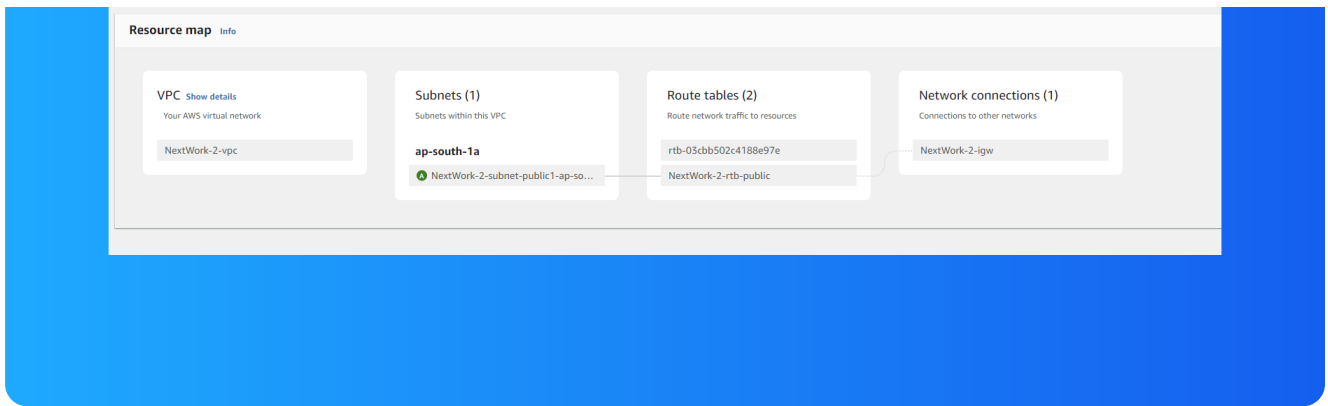
I started my project by launching 2 VPCs, one with the CIDR block 10.1.0.0/16 and the other with 10.2.0.0/16. I created 1 public subnet in each VPC.

The CIDR blocks for VPCs 1 and 2 are 10.1.0.0/16 and 10.2.0.0/16, respectively. They have to be unique because overlapping IP address ranges would cause routing issues, preventing the VPCs from communicating effectively or creating conflicts in IP.

I also launched 2 EC2 instances

I didn't set up key pairs for these EC2 instances as I plan to use them solely for testing the VPC peering connection, and I can access them through the console or another method without needing SSH access at this stage.





ajv.singh3107@gmail.com
NextWork Student

[NextWork.org](https://nextwork.org)

VPC Peering

A VPC peering connection is a private network link between two VPCs that allows them to communicate directly without using the internet. It enables secure, low-latency traffic flow between VPCs by updating routing tables and security settings.

VPCs would use peering connections to securely share resources like EC2 instances or databases across different VPCs, reduce latency by avoiding internet traffic, and enable communication between environments such as development and production.

The difference between a Requester and an Acceptor in a peering connection is that the Requester initiates the peering connection request between two VPCs, while the Acceptor approves or denies the request, allowing the connection to be established.

Select another VPC to peer with

Account

☒ My account
☐ Another account

Region

☒ This Region (ap-south-1)
☐ Another Region

VPC ID (Accepted)

vpc-0c124528563d68536 (NextWork-2-vpc)

VPC CIDRs for vpc-0c124528563d68536 (NextWork-2-vpc)

CIDR	Status	Status reason
10.2.0.0/16	Associated	-

Tags

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

Key	Value - optional
Name	VPC 1 <-> VPC 2

Add new tag

You can add 49 more tags.

Cancel Create peering connection



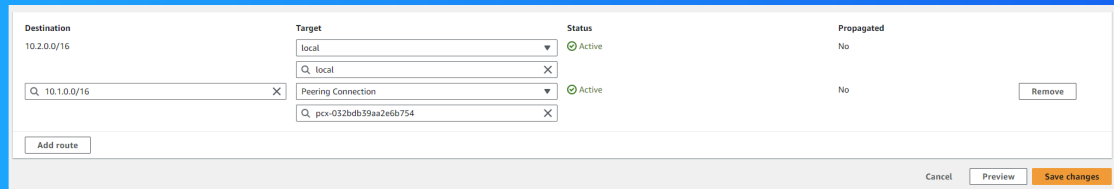
ajv.singh3107@gmail.com
NextWork Student

[NextWork.org](https://nextwork.org)

Updating route tables

After accepting a peering connection, my VPCs' route tables need to be updated because they must know how to direct traffic to the other VPC. Adding a new route ensures that traffic can flow between the two VPCs over the peering connection.

My VPCs' new routes have a destination of the CIDR block of the other VPC. The routes' target was the VPC peering connection ID to enable traffic flow between the two VPCs.



The screenshot shows the AWS VPC Peering console. It displays a table with columns: Destination, Target, Status, and Propagated. The first row shows a peering connection between two VPCs. The Destination is 10.2.0.0/16, the Target is local, the Status is Active, and the Propagated status is No. The second row shows a peering connection between two VPCs. The Destination is 10.10.0.0/16, the Target is Peering Connection, the Status is Active, and the Propagated status is No. There is a Remove button next to the second row. At the bottom, there are buttons for Add route, Cancel, Preview, and Save changes.

Destination	Target	Status	Propagated
10.2.0.0/16	local	Active	No
10.10.0.0/16	Peering Connection	Active	No

Buttons: Add route, Cancel, Preview, Save changes, Remove



ajv.singh3107@gmail.com
NextWork Student

[NextWork.org](https://nextwork.org)

In the second part of my project...

Step 5 - Use EC2 Instance Connect

We're using EC2 Instance Connect to access your first EC2 instance. This allows you to interact with the instance through the terminal. If you encounter a connection error, you'll troubleshoot and resolve it to ensure successful access to the instance.

Step 6 - Connect to EC2 Instance 1

We're using EC2 Instance Connect again to connect to Instance 1. After addressing the previous error, we'll attempt to establish the connection once

more. If another error occurs, we'll troubleshoot and fix it to gain access to the instance success

Step 7 - Test VPC Peering

We're getting Instance 1 to send test messages to Instance 2 to verify the VPC peering connection. We'll troubleshoot any connection errors to ensure Instance 2 can receive and respond to the messages from Instance 1.



ajv.singh3107@gmail.com
NextWork Student

[NextWork.org](https://nextwork.org)

Troubleshooting Instance Connect

I used EC2 Instance Connect to access my first EC2 instance quickly without needing a key pair. It provides a convenient way to establish a connection directly from the AWS Management Console, simplifying the login process for testing.

I was stopped from using EC2 Instance Connect as the instance's security group did not allow incoming SSH traffic on port 22, preventing me from establishing a connection to the instance.

Connect to instance Info

Connect to your instance i-0b37d43d03643bde3 (Instance - NextWork VPC 1) using any of these options

EC2 Instance Connect

Session Manager

SSH client

EC2 serial console



No public IPv4 or IPv6 address assigned

With no public IPv4 or IPv6 address, you can't use EC2 Instance Connect. Alternatively, you can try connecting using [EC2 Instance Connect Endpoint](#).



Instance is not in public subnet

Associated subnet [subnet-0c3bcb1989bb92fd8 \(NextWork-1-subnet-public1-ap-south-1a\)](#) is not a public subnet.

To use EC2 Instance Connect, your instance must be in a public subnet. [To make the subnet a public subnet](#), add a route in the subnet route table to an internet gateway.



ajv.singh3107@gmail.com

NextWork Student

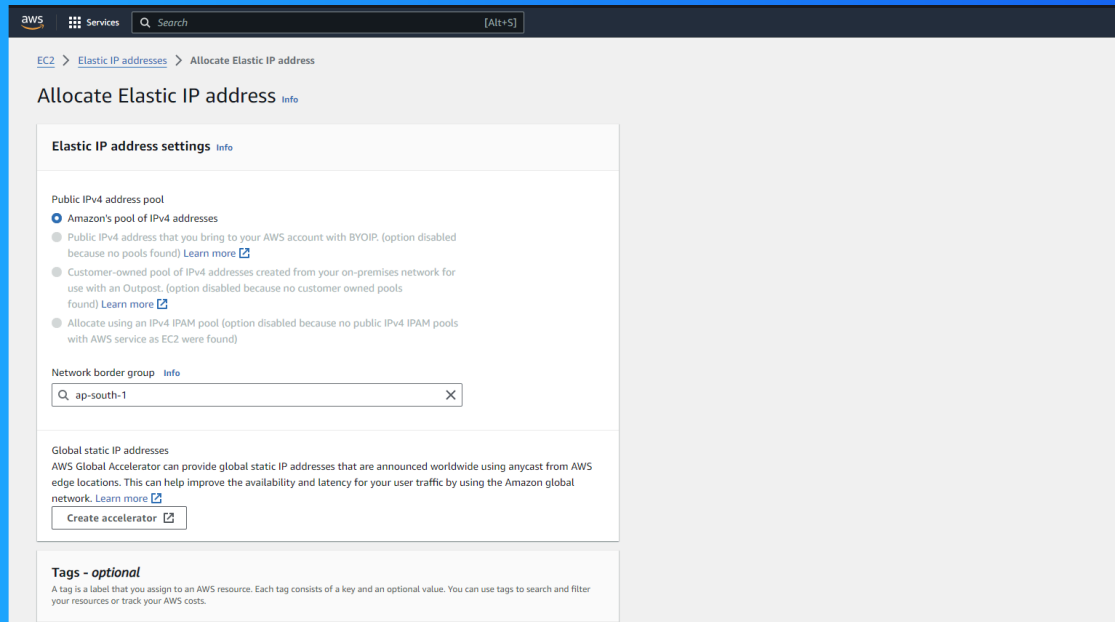
[NextWork.org](https://nextwork.org)

Elastic IP addresses

To resolve this error, I set up Elastic IP addresses. Elastic IP addresses are static, public IPs associated with EC2 instances. They ensure consistent connectivity, allowing access to instances even if the underlying instance changes or is restarted

Associating an Elastic IP address resolved the error because it provided a static public IP for the EC2 instance. This allowed me to bypass security group

restrictions and ensure consistent connectivity for accessing the instance via EC2 connect.



ajv.singh3107@gmail.com
NextWork Student

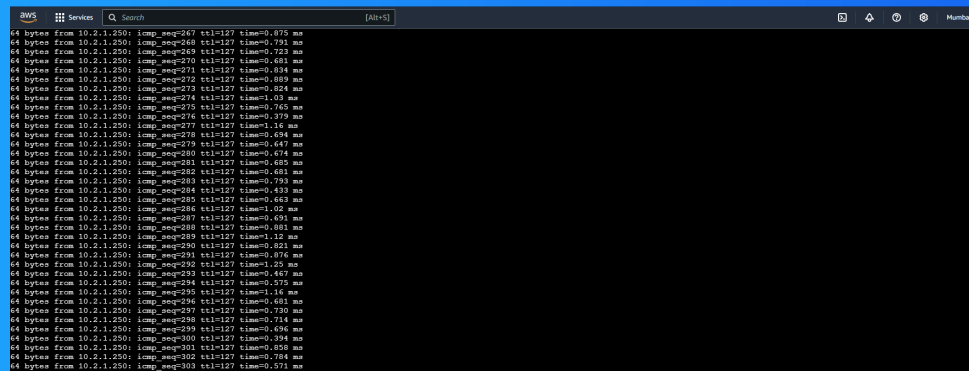
[NextWork.org](https://nextwork.org)

Troubleshooting ping issues

To test VPC peering, I ran the command ``ping <Instance 2's private IP address>``. This helped verify the connectivity between Instance 1 and Instance 2 over the peering connection.

A successful ping test would validate my VPC peering connection because it shows that Instance 1 can reach Instance 2 over the private network. This confirms that routing and security settings are properly configured for communication between the VPC

I had to update my second EC2 instance's security group because it wasn't allowing inbound traffic from Instance 1. I added a new rule that permitted ICMP traffic from the CIDR block of the first VPC to enable ping responses.



```
aws
Services
Search
[Alt+S]
Mumbai
64 bytes from 10.2.1.250: icmp_seq=267 ttl=127 time=0.875 ms
64 bytes from 10.2.1.250: icmp_seq=268 ttl=127 time=0.791 ms
64 bytes from 10.2.1.250: icmp_seq=269 ttl=127 time=0.723 ms
64 bytes from 10.2.1.250: icmp_seq=270 ttl=127 time=0.681 ms
64 bytes from 10.2.1.250: icmp_seq=271 ttl=127 time=0.834 ms
64 bytes from 10.2.1.250: icmp_seq=272 ttl=127 time=0.889 ms
64 bytes from 10.2.1.250: icmp_seq=273 ttl=127 time=0.828 ms
64 bytes from 10.2.1.250: icmp_seq=274 ttl=127 time=1.03 ms
64 bytes from 10.2.1.250: icmp_seq=275 ttl=127 time=0.765 ms
64 bytes from 10.2.1.250: icmp_seq=276 ttl=127 time=0.379 ms
64 bytes from 10.2.1.250: icmp_seq=277 ttl=127 time=0.116 ms
64 bytes from 10.2.1.250: icmp_seq=278 ttl=127 time=0.694 ms
64 bytes from 10.2.1.250: icmp_seq=279 ttl=127 time=0.647 ms
64 bytes from 10.2.1.250: icmp_seq=280 ttl=127 time=0.674 ms
64 bytes from 10.2.1.250: icmp_seq=281 ttl=127 time=0.685 ms
64 bytes from 10.2.1.250: icmp_seq=282 ttl=127 time=0.681 ms
64 bytes from 10.2.1.250: icmp_seq=283 ttl=127 time=0.793 ms
64 bytes from 10.2.1.250: icmp_seq=284 ttl=127 time=0.433 ms
64 bytes from 10.2.1.250: icmp_seq=285 ttl=127 time=0.443 ms
64 bytes from 10.2.1.250: icmp_seq=286 ttl=127 time=1.02 ms
64 bytes from 10.2.1.250: icmp_seq=287 ttl=127 time=0.631 ms
64 bytes from 10.2.1.250: icmp_seq=288 ttl=127 time=0.881 ms
64 bytes from 10.2.1.250: icmp_seq=289 ttl=127 time=0.112 ms
64 bytes from 10.2.1.250: icmp_seq=290 ttl=127 time=0.621 ms
64 bytes from 10.2.1.250: icmp_seq=291 ttl=127 time=0.876 ms
64 bytes from 10.2.1.250: icmp_seq=292 ttl=127 time=1.25 ms
64 bytes from 10.2.1.250: icmp_seq=293 ttl=127 time=0.467 ms
64 bytes from 10.2.1.250: icmp_seq=294 ttl=127 time=0.575 ms
64 bytes from 10.2.1.250: icmp_seq=295 ttl=127 time=1.16 ms
64 bytes from 10.2.1.250: icmp_seq=296 ttl=127 time=0.681 ms
64 bytes from 10.2.1.250: icmp_seq=297 ttl=127 time=0.730 ms
64 bytes from 10.2.1.250: icmp_seq=298 ttl=127 time=0.714 ms
64 bytes from 10.2.1.250: icmp_seq=299 ttl=127 time=0.636 ms
64 bytes from 10.2.1.250: icmp_seq=300 ttl=127 time=0.394 ms
64 bytes from 10.2.1.250: icmp_seq=301 ttl=127 time=0.826 ms
64 bytes from 10.2.1.250: icmp_seq=302 ttl=127 time=0.784 ms
64 bytes from 10.2.1.250: icmp_seq=303 ttl=127 time=0.571 ms
```



[NextWork.org](https://learn.nextwork.org)

Everyone should be in a job they love.

Check out nextwork.org for
more projects

